ChatGPT said:

Serialize and Deserialize Binary Trees — Notes

1. Introduction

In a tree data structure, **Serialization and Deserialization** are techniques used to convert a binary tree into a format that can be stored or transmitted and then reconstruct the original tree.

These techniques are commonly used in data structures, file storage, databases, and coding interviews.

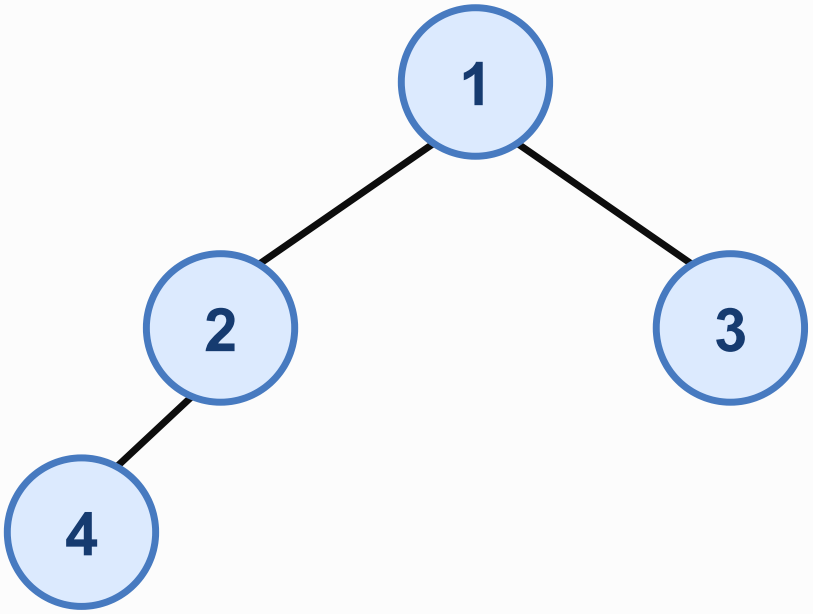
2. Serialization

Serialization is the process of converting a binary tree into a string or sequence of data.

During serialization, the values of the nodes and information about the structure of the tree are stored in a specific order. Null or missing children must also be represented so that the original tree can be reconstructed correctly.

Example

Consider the following binary tree:



Binary tree with nodes 1, 2, 3 and 4

Using preorder traversal (Root, Left, Right) and marking missing children as `N`, the serialized representation is:

```
1,2,4,N,N,N,3,N,N
```

Here, `N` represents a null or missing node.

3. Deserialization

Deserialization is the process of converting the serialized data back into the original binary tree.

It reads the stored values in the same order and recreates each node, including its left and right children. When it encounters `N`, it creates no node at that position.

For example, the serialized string

`1,2,4,N,N,N,3,N,N`

can be converted back into the original tree shown above.

4. Difference Between Serialization and Deserialization

Serialization	Deserialization
Converts a tree into a string or sequence.	Converts a string or sequence into a tree.
Used to store or transmit tree data.	Used to reconstruct tree data.
Traverses the tree and records node values and null markers.	Reads the recorded data and creates nodes.
It is the encoding process.	It is the decoding process.

5. Algorithms

A. Serialization Algorithm

1. Start from the root node.
2. Traverse the tree using preorder traversal.
3. Store each node's value.
4. If a node is null, store a null marker such as `N`.
5. Continue until the entire tree structure has been recorded.
6. Return the resulting string.

B. Deserialization Algorithm

1. Read the serialized string.
2. Split the string into individual values.
3. Read the next value.
4. If the value is `N`, return `NULL`.
5. Otherwise, create a new node with that value.
6. Recursively construct its left and right subtrees.
7. Return the root of the reconstructed tree.

6. Time and Space Complexity

Let n be the number of nodes in the binary tree.

Operation	Time Complexity	Space Complexity
Serialization	$O(n)$	$O(n)$
Deserialization	$O(n)$	$O(n)$

Both operations require linear time because each node and each null position is processed a constant number of times.

7. Applications

1. Storing binary trees in files or databases.
2. Transmitting tree structures over a network.
3. Saving and restoring data structures.
4. Implementing tree-based coding interview problems.
5. Copying or reconstructing a tree in another program.

8. Conclusion

Serialization and deserialization are important operations for storing and reconstructing binary trees. Serialization converts a tree into a sequence of data, while deserialization reconstructs the original tree from that sequence. Null markers are important because they preserve the structure of the tree.

Important exam point: Serialization stores both node values and structural information, while deserialization uses that information to recreate the original binary tree.